과제 발표

Verilog microwave 구현하기

김형진

목차

01
과제 개요

02
과제 구현 내용

03
트러블 슈팅

04
시연영상

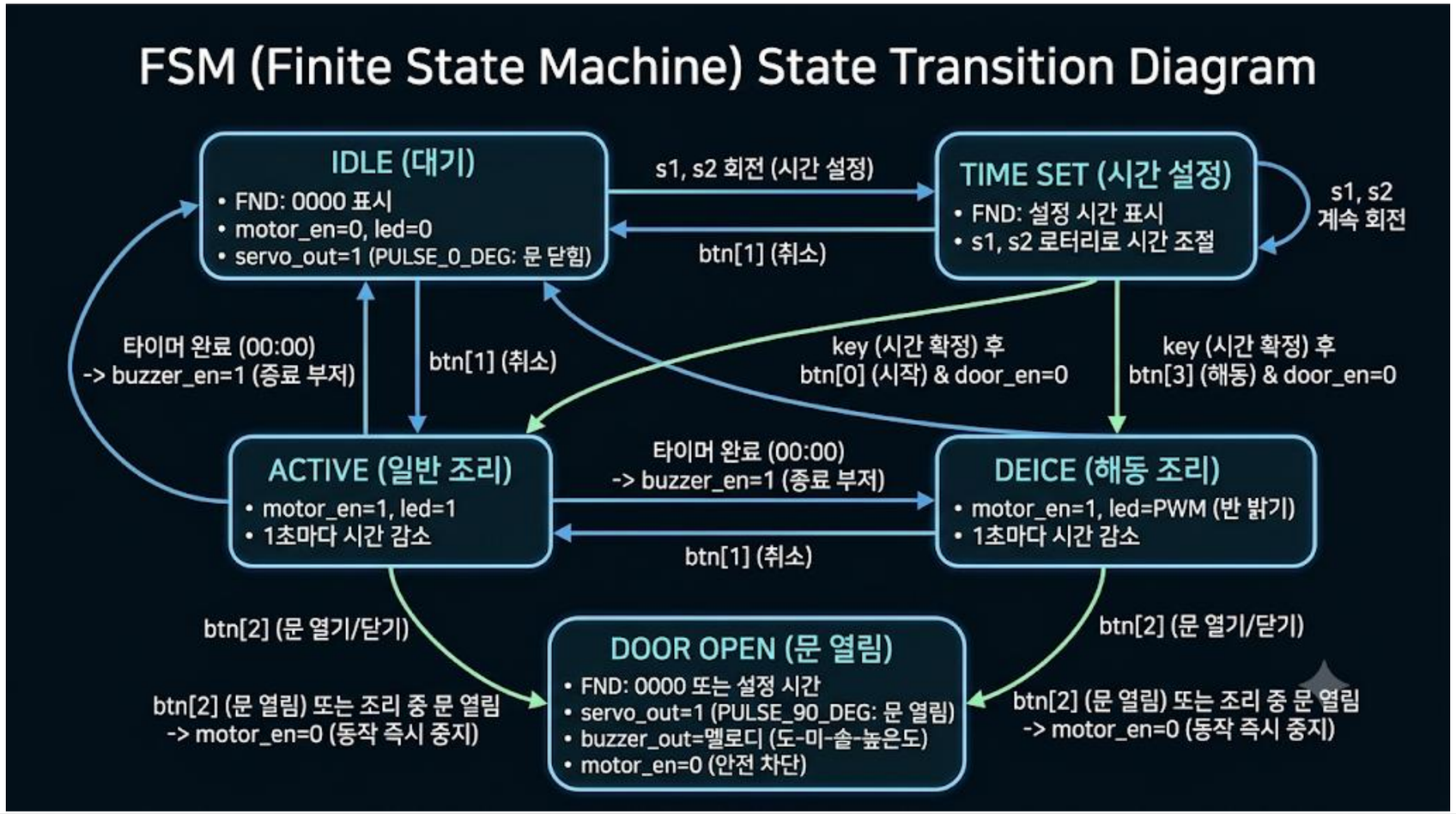
과제 개요

- 과제명:
  - FPGA 기반 스마트 전자레인지 제어 시스템

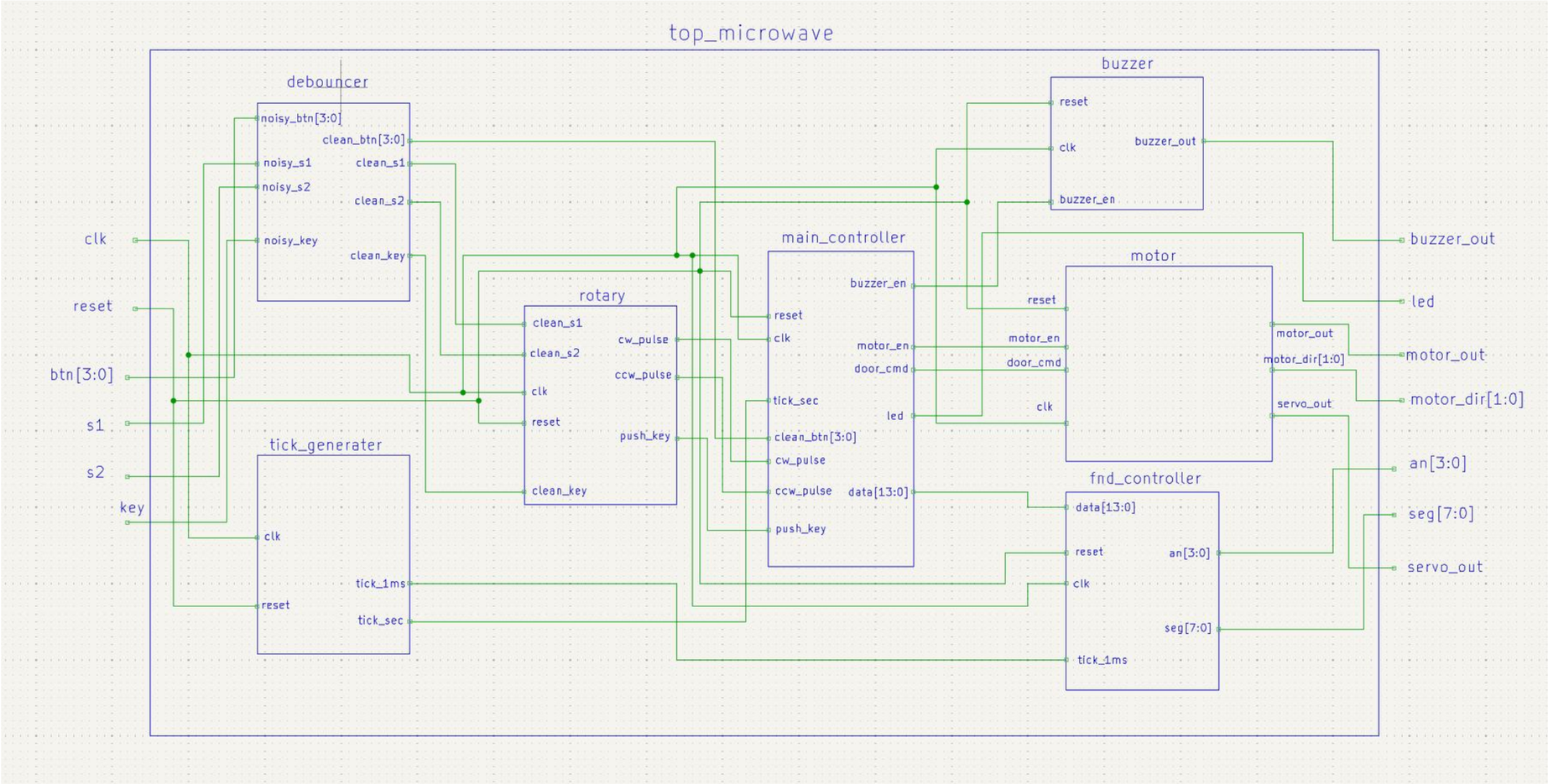
- 목표:
  - 디지털 논리 회로를 이용해 실제 전자레인지의 동작 흐름(시간 설정, 조리, 해동, 안전장치)을 설계
  - PWM 및 타이머를 활용하여 주변 하드웨어를 정밀하게 제어하는 통합 컨트롤러를 구현

- 주요 개발 내용:
  - 입력 신호 처리
  - 정밀 타이머 및 디스플레이
  - 도어 제어: 20ms 주기의 PWM 신호를 생성하여 서보모터의 타겟 펄스 폭(1.0ms/1.5ms)을 조절,
  - 문의 개폐 각도와 유지 토크 정밀 제어.

과제 구현 내용


FSM (Finite State Machine) State Transition Diagram
IDLE (대기)
• FND: 0000 표시
• motor_en=0, led=0
• servo_out=1 (PULSE_0_DEG: 문 닫힘)
s1, s2 회전 (시간 설정)
btn[1] (취소)
TIME SET (시간 설정)
• FND: 설정 시간 표시
• s1, s2 로터리로 시간 조절
s1, s2
계속 회전
타이머 완료 (00:00)
-> buzzer_en=1 (종료 부저)
btn[1] (취소)
key (시간 확정) 후
btn[0] (시작) & door_en=0
key (시간 확정) 후
btn[3] (해동) & door_en=0
ACTIVE (일반 조리)
• motor_en=1, led=1
• 1초마다 시간 감소
타이머 완료 (00:00)
-> buzzer_en=1 (종료 부저)
btn[1] (취소)
DEICE (해동 조리)
• motor_en=1, led=PWM (반 밝기)
• 1초마다 시간 감소
btn[2] (문 열기/닫기)
btn[2] (문 열기/닫기)
DOOR OPEN (문 열림)
• FND: 0000 또는 설정 시간
• servo_out=1 (PULSE_90_DEG: 문 열림)
• buzzer_out=멜로디 (도-미-솔-높은도)
• motor_en=0 (안전 차단)
btn[2] (문 열림) 또는 조리 중 문 열림
-> motor_en=0 (동작 즉시 중지)
btn[2] (문 열림) 또는 조리 중 문 열림
-> motor_en=0 (동작 즉시 중지)

과제 구현 내용


top_microwave
debouncer
noisy_btn[3:0]
clean_btn[3:0]
noisy_s1
clean_s1
noisy_s2
clean_s2
noisy_key
clean_key
rotary
clean_s1
clean_s2
clk
reset
cw_pulse
ccw_pulse
push_key
clean_key
tick_generater
clk
reset
tick_1ms
tick_sec
main_controller
reset
clk
tick_sec
clean_btn[3:0]
cw_pulse
ccw_pulse
push_key
buzzer_en
motor_en
door_cmd
led
data[13:0]
buzzer
reset
clk
buzzer_en
buzzer_out
motor
reset
motor_en
door_cmd
clk
motor_out
motor_dir[1:0]
servo_out
fnd_controller
data[13:0]
reset
clk
tick_1ms
an[3:0]
seg[7:0]
clk
reset
btn[3:0]
s1
s2
key
buzzer_out
led
motor_out
motor_dir[1:0]
an[3:0]
seg[7:0]
servo_out

과제 구현 내용

rotary

```
module rotary(
    input reset, clk, s1, s2,
    output reg cw_pulse, ccw_pulse
    );

    reg [1:0] r_direction = 2'b00; // 시계 01 반시계 10
    reg [1:0] r_prev_state = 2'b00;
    reg [1:0] r_curr_state = 2'b00;
    reg [1:0] r_step = 2'b00;
    reg [7:0] r_counter = 8'h00;
    //1. s1 s2 로직 체크 (순차회로)
    //방향 설정
    //CW  00 --> 10 --> 11 --> 01 --> 00 (1씩 증가 r_step)
    //CCW 00 --> 01 --> 11 --> 10 --> 00
    always @(posedge clk, posedge reset) begin
        if(reset)begin
            cw_pulse <=0;
            ccw_pulse <=0;
            r_prev_state <= 2'b00;
            r_curr_state <= 2'b00;
            r_step <= 2'b00;
            r_counter <= 8'h00;
        end else begin
            r_prev_state <= r_curr_state;
            cw_pulse <=0;
            ccw_pulse <=0;
            r_curr_state <= {s1, s2};
            case ({r_prev_state, r_curr_state})
                4'b0010,4'b1011,4'b1101,4'b0100 : begin //cw
                    if(r_counter < 8'hFF && r_step == 2'b11) begin
                        cw_pulse <= 1;
                        r_counter <= r_counter +1;
                    end
                    r_step <= r_step + 1;
                end
                4'b0001,4'b0111,4'b1110,4'b1000 : begin //ccw
                    if(r_counter > 8'h00 && r_step == 2'b11) begin
                        ccw_pulse <= 1;
                        r_counter <= r_counter -1;
                    end
                    r_step <= r_step +1;
                end
            endcase
        end
    end
```

- 한 펄스 움직일때 마다 펄스발생 → 시간추가로직

과제 구현 내용

main_controller

```
// [A] 대기 상태 (전자레인지 정지 중)
if(!motor_en) begin
    // 엔코더 회전: 입력 시간 조절 (10초 단위)
    if(cw_pulse) begin
        time_set_done <= 1'b0;
        if(temp_sec == 6'd50) begin
            temp_sec <= 6'd0;
            temp_min <= temp_min + 1'b1;
        end else begin
            temp_sec <= temp_sec + 6'd10;
        end
    end
    else if(ccw_pulse) begin
        time_set_done <= 1'b0;
        if(temp_sec == 6'd0) begin
            if(temp_min > 0) begin
                temp_sec <= 6'd50;
                temp_min <= temp_min - 1'b1;
            end
        end else begin
            temp_sec <= temp_sec - 6'd10;
        end
    end

    // 엔코더 버튼 누름: 시간 최종 확정
    if(w_push_key) begin
        if(temp_min > 0 || temp_sec > 0) begin
            r_min_counter <= temp_min;
            r_sec_counter <= temp_sec;
            time_set_done <= 1'b1;
        end
    end
```

```
// [B] 동작 상태 (모터 구동 중: 1초마다 다운카운트)
else begin
    if(tick_sec) begin
        if(r_sec_counter == 6'd0) begin
            if(r_min_counter > 0) begin
                r_sec_counter <= 6'd59;
                r_min_counter <= r_min_counter - 1'b1;
            end
        end else begin
            r_sec_counter <= r_sec_counter - 1'b1;
        end
    end
end
```

- CW방향 한 펄스 → 10초씩 추가
- CCW방향 한펄스 추가 → 10초 감소

과제 구현 내용

main_controller

```
// 1. 강제 정지 버튼 또는 조리 도중 문이 열렸을 때 즉시 모터 정지
if(w_btn_stop || (motor_en && door_en)) begin
    motor_en  <= 1'b0;
    r_mode    <= 2'b00;
    buzzer_en <= 1'b0;
end
// 2. 조리 시간 종료 시 자동 정지 & 부저 울림
else if(motor_en && (r_min_counter == 6'd0) && (r_sec_counter == 6'd0)) begin
    motor_en  <= 1'b0;
    r_mode    <= 2'b00;
    buzzer_en <= 1'b1; // 완료 알림음
end
// 3. 일반 시작 (시간 확정 상태 + 문이 닫혀 있을 때만)
else if(w_btn_start && time_set_done && !door_en) begin
    motor_en  <= 1'b1;
    r_mode    <= 2'b01;
    buzzer_en <= 1'b0;
end
// 4. 해동 시작 (시간 확정 상태 + 문이 닫혀 있을 때만)
else if(w_btn_deice && time_set_done && !door_en) begin
    motor_en  <= 1'b1;
    r_mode    <= 2'b10;
    buzzer_en <= 1'b0;
end
```

버튼 동작 구현

- btn[0] → 일반 조리 시작
- btn[1] → 취소 및 일시정지
- btn[2] → 문열고 닫기
- btn[3] → 해동모드 시작

```
// 4. LED PWM 밝기 제어
reg [7:0] pwm_cnt;

always @(posedge clk or posedge reset) begin
    if(reset) begin
        pwm_cnt <= 8'd0;
    end else begin
        pwm_cnt <= pwm_cnt + 1'b1;
    end
end

always @(*) begin
    case(r_mode)
        2'b01:   led = 1'b1;              // 일반: 최대 밝기
        2'b10:   led = (pwm_cnt < 8'd128);// 해동: 반 밝기 (50%)
        default: led = 1'b0;              // 정지: 소등
    endcase
end


// 5. FND 표시 데이터 (설정 중에는 temp 표시, 동작 중에는 남은 시간 표시)
wire [5:0] disp_min = motor_en ? r_min_counter : (time_set_done ? r_min_counter : temp_min);
wire [5:0] disp_sec = motor_en ? r_sec_counter : (time_set_done ? r_sec_counter : temp_sec);

assign data = (disp_min * 14'd100) + disp_sec;
```

LED 구현

- 일반 조리 모드 → 100% duty
- 해동 모드 → 50% duty

과제 구현 내용

motor

```
parameter DC_PERIOD = 14'd10_000; // 100MHz / 10kHz = 10,000
parameter DC_DUTY   = 14'd10_000;  // 100% 듀티

reg [13:0] dc_counter;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        dc_counter <= 14'd0;
    end else if (motor_en) begin
        if (dc_counter >= DC_PERIOD - 1) begin
            dc_counter <= 14'd0;
        end else begin
            dc_counter <= dc_counter + 1'b1;
        end
    end else begin
        dc_counter <= 14'd0;
    end
end

// motor_en이 1일 때만 PWM 출력
assign motor_out = motor_en ? (dc_counter < DC_DUTY) : 1'b0;
// L298N/드라이버 방향 신호 (1이면 정회전, 0이면 정지)
assign motor_dir = motor_en ? 2'b10 : 2'b00;
```

모터 동작 구현

- 전자렌지 바닥 판 모터 구현
- 모터 출력이 약한 관계로 100% 듀티로 출력

과제 구현 내용

motor

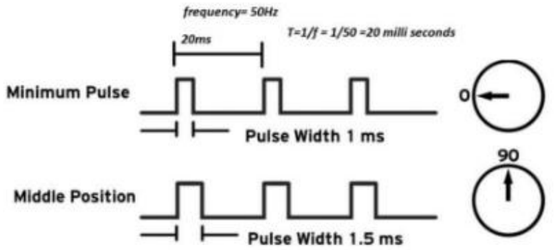
```
parameter SERVO_PERIOD = 21'd2_000_000; // 20ms
parameter PULSE_0_DEG  = 21'd100_000;   // 1.0ms (0도: 문 닫힘)
parameter PULSE_90_DEG = 21'd150_000;   // 1.5ms (90도: 문 열림)
```

```
always @(posedge clk or posedge reset) begin
    if (reset) begin
        servo_counter  <= 21'd0;
        move_counter   <= 32'd0;
        r_prev_door_en <= 1'b0;
    end else begin
        // 1. 서보 PWM 주기 카운터 (계속 돎)
        if (servo_counter >= SERVO_PERIOD - 1) begin
            servo_counter <= 21'd0;
        end else begin
            servo_counter <= servo_counter + 1'b1;
        end

        // 2. 문 열림/닫힘 변화 감지 및 동작 타이머 구동
        r_prev_door_en <= door_en;
```

```
wire [20:0] target_pulse = door_en ? PULSE_90_DEG : PULSE_0_DEG;

// 타이머가 돌아가는 동작 중에만 파형 출력, 끝나면 0 출력
assign servo_out = (move_counter > 0) ? (servo_counter < target_pulse) : 1'b0;
```



서보모터 동작 구현

- 전자렌지 문 서보 모터 구현
- 20ms 주기를 가지도록 구현
- 0도 에서는 펄스 폭이 1ms
- 90도에서는 펄스폭이 1.5ms

과제 구현 내용

buzzer

```
// buzzer_en 시작 엣지 검출 (신호가 들어올 때 1회 재생 시작)
reg r_prev_buzzer_en;
wire w_start_beep = (buzzer_en && !r_prev_buzzer_en);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        r_prev_buzzer_en <= 1'b0;
        count_70ms_1     <= 24'd0;
        count_step       <= 3'd4; // 4는 정지(IDLE) 상태
        r_khz_count      <= 16'd0;
        r_beep_tone      <= 1'b0;
    end else begin
        r_prev_buzzer_en <= buzzer_en;

        // 시작 신호가 들어오면 0단계부터 재생 트리거
        if (w_start_beep) begin
            count_step   <= 3'd0;
            count_70ms_1 <= 24'd0;
            r_khz_count  <= 16'd0;
            r_beep_tone  <= 1'b0;
        end
        // 연주 중 (0 ~ 3단계)
        else if (count_step < 3'd4) begin
            if (count_70ms_1 >= ms_70 - 1) begin
                count_70ms_1 <= 24'd0;
                count_step   <= count_step + 1'b1; // 4가 되면 다음 클럭부터 정지
            end else begin
                count_70ms_1 <= count_70ms_1 + 1'b1;
            end

            // 음계 주파수 토글
            case (count_step)
                3'd0: begin
                    if (r_khz_count >= duty_1khz - 1) begin
                        r_khz_count <= 16'd0;
                        r_beep_tone <= ~r_beep_tone;
                    end else r_khz_count <= r_khz_count + 1'b1;
                end
                3'd1: begin
                    if (r_khz_count >= duty_2khz - 1) begin
                        r_khz_count <= 16'd0;
                        r_beep_tone <= ~r_beep_tone;
                    end else r_khz_count <= r_khz_count + 1'b1;
                end
                3'd2: begin
                    if (r_khz_count >= duty_3khz - 1) begin
                        r_khz_count <= 16'd0;
                        r_beep_tone <= ~r_beep_tone;
                    end else r_khz_count <= r_khz_count + 1'b1;
                end
                3'd3: begin
                    if (r_khz_count >= duty_4khz - 1) begin
                        r_khz_count <= 16'd0;
                        r_beep_tone <= ~r_beep_tone;
                    end else r_khz_count <= r_khz_count + 1'b1;
                end
                default: r_beep_tone <= 1'b0;
            endcase
        end else begin
            r_beep_tone <= 1'b0; // 연주 종료 후 소등
        end
    end
end
```

```
reg r_prev_door_en;
// wire w_start_door = (door_en && !r_prev_door_en);
wire w_start_door = (door_en != r_prev_door_en);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        r_prev_door_en <= 1'b0;
        count_70ms_2   <= 24'd0;
        next_melody    <= 3'd4;
        r_clk_count    <= 22'd0;
        r_door_tone    <= 1'b0;
    end else begin
        r_prev_door_en <= door_en;

        if (w_start_door) begin
            next_melody  <= 3'd0;
            count_70ms_2 <= 24'd0;
            r_clk_count  <= 22'd0;
            r_door_tone  <= 1'b0;
        end
        else if (next_melody < 3'd4) begin
            if (count_70ms_2 >= ms_70 - 1) begin
                count_70ms_2 <= 24'd0;
                next_melody  <= next_melody + 1'b1;
            end else begin
                count_70ms_2 <= count_70ms_2 + 1'b1;
            end

            case (next_melody)
                3'd0: begin
                    if (r_clk_count >= DO - 1) begin
                        r_clk_count <= 22'd0;
                        r_door_tone <= ~r_door_tone;
                    end else r_clk_count <= r_clk_count + 1'b1;
                end
                3'd1: begin
                    if (r_clk_count >= MI - 1) begin
                        r_clk_count <= 22'd0;
                        r_door_tone <= ~r_door_tone;
                    end else r_clk_count <= r_clk_count + 1'b1;
                end
                3'd2: begin
                    if (r_clk_count >= SOL - 1) begin
                        r_clk_count <= 22'd0;
                        r_door_tone <= ~r_door_tone;
                    end else r_clk_count <= r_clk_count + 1'b1;
                end
                3'd3: begin
                    if (r_clk_count >= DDO - 1) begin
                        r_clk_count <= 22'd0;
                        r_door_tone <= ~r_door_tone;
                    end else r_clk_count <= r_clk_count + 1'b1;
                end
                default: r_door_tone <= 1'b0;
            endcase
        end else begin
            r_door_tone <= 1'b0;
        end
    end
end
```

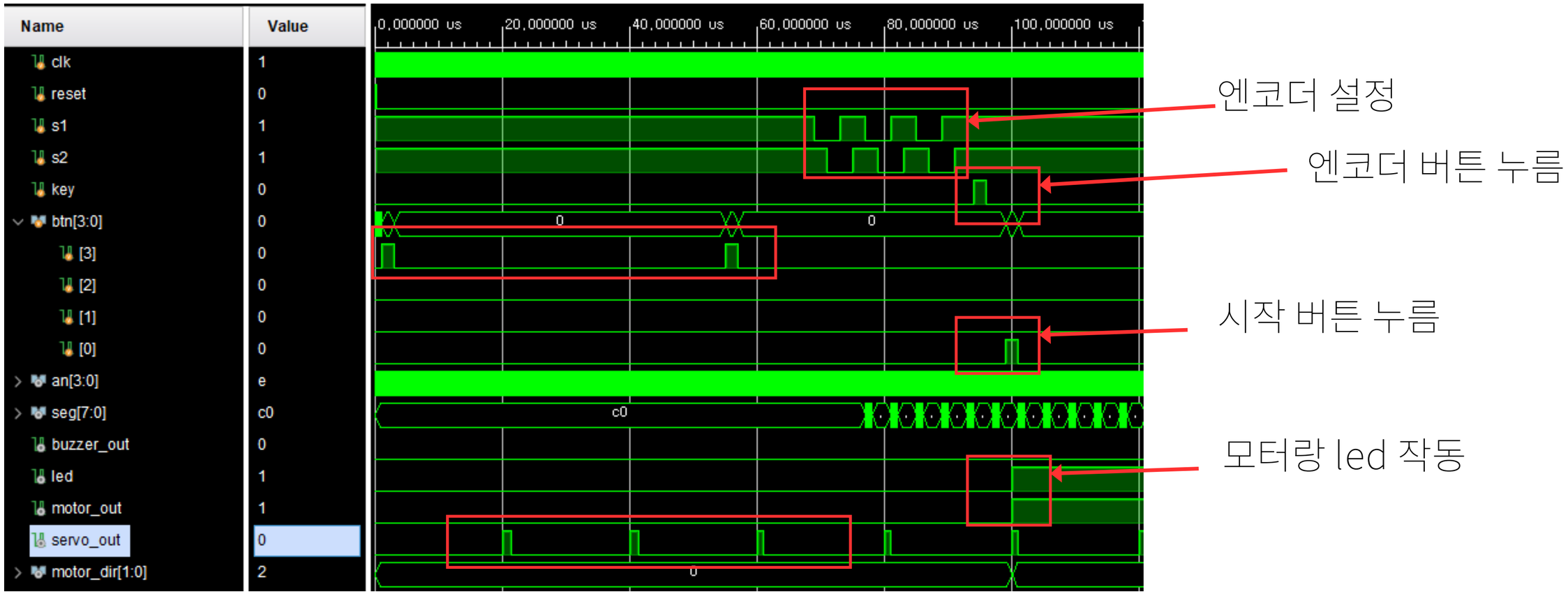
부터 동작 구현

- 전자레인지 조리가 완료되었을 때 → 삐삐삐
- 문열고 닫을 때 → 도미솔도~

buzzer_en → 조리 타이머가 완료되었다면 1 평상시 0

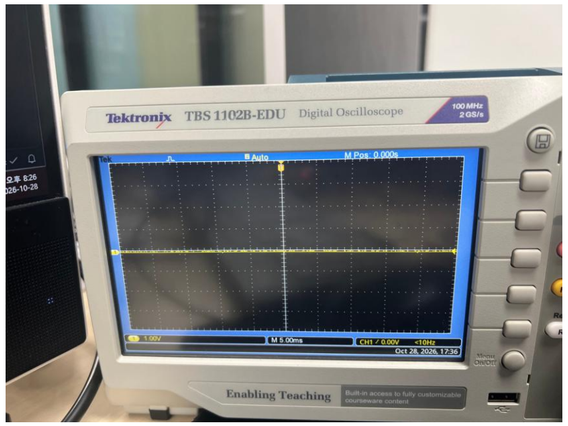
door_en → 문을 열었다면 1 닫았다면 0

과제 구현 내용(테스트벤치)

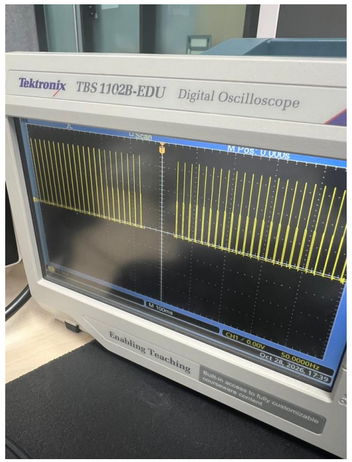


트러블 슈팅

1. 문열기 버튼을 눌러도 서보모터가 작동을 안하는 문제



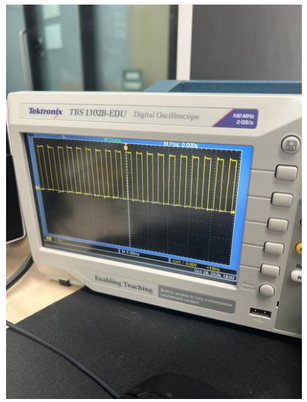
JB 포트 A15 핀 파형



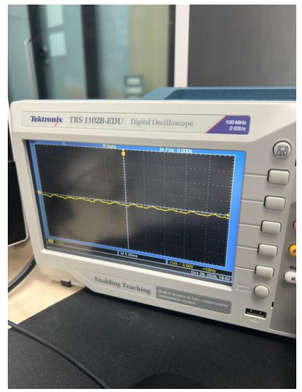
JB 포트 A14 핀 파형

- 문제 : 문열기 버튼을 눌러도 파형이 나오지 않음
- 원인 : 하드웨어 핀 설정을 잘못하고 있었음 (14번에 연결했음)

2. 문 열때는 부저가 울리지만 닫을 때 안울리는 문제 발생



열기 버튼 눌렀을 때



닫기 버튼 눌렀을 때

```
// wire w_start_door = (door_en && !r_prev_door_en);
wire w_start_door = (door_en != r_prev_door_en);
```

- 문제 : door_en은 누를때마다 토글되는 신호라서 상승에지만 검출
- 원인 : 버튼 처럼 이전 클럭을 저장해서 상승에지를 검출하는 신호를 만듬
- 문이 열리면 1 문이 닫히면 0으로 오는 신호라서 하강에지도 같이 검출해줘야 함

시연 영상

